

MC-PILOT Throwing Arm — Methods, Rationale & Evidence

Headline result: after diagnosing and fixing two silent failure modes, the trained policy achieves **250/250 hits** on fresh evaluation targets (5 seeds × 50 targets), mean landing error **1.86 cm**, worst throw **3.81 cm** — at both the 10 cm and the tight 5 cm thresholds.

1. Component map — what we used, why, how

Component	Why this choice	How we used it
Gaussian Process dynamics model (3 independent sparse GPs, squared-exponential kernel)	GPs are the only model class that gives calibrated uncertainty from ~5 data points. A neural network would need hundreds of throws; the GP's smoothness prior does the heavy lifting where data is scarce.	Input: 6-D ball state $[x,y,z,vx,vy,vz]$. Output: one GP per velocity-correction component (Δvx , Δvy , Δvz). Retrained after every throw via Adam on marginal log-likelihood; subset-of-data approximation keeps inference cheap.
Single-shot RBF policy (250 basis functions, tanh-squashed output)	The throw is decided entirely at release — so the policy fires once per throw instead of every timestep. This collapses the search space and is the single biggest reason ~10 trials suffice (vs. dozens for step-by-step control).	Maps target $(Px, Py) \rightarrow$ scalar release speed in $[0, uM]$ via $(uM/2)(\tanh(\cdot)+1)$. Trainable parameters: 250 center weights + lengthscales.
Monte Carlo policy gradient (M=400 particles, reparameterization trick)	The initial velocity is a nonlinear function of the target, so the state distribution is non-Gaussian from $t=0$ — PILCO-style analytic moment matching would be biased immediately. Sampling particles propagates the true distribution with no Gaussian assumption.	Each particle draws $\epsilon \sim N(0, I)$ once, flows through the GP posterior as $\mu(x) + L(x)\epsilon$; gradients flow through the whole rollout into policy weights via autodiff. Adam, up to 1500 steps/trial. Gradient noise shrinks as $O(1/\sqrt{M})$.
Stratified exploration (speed bands)	Random exploration can, by luck, sample only part of the speed range — the GP never sees fast-throw physics and the policy converges to a wrong optimum <i>silently</i> . Observed: 3 of 5 seeds fail this way.	$[0, uM]$ split into $N_{exp}=5$ equal bands; exploration throw i samples from band i . Guarantees full-range GP coverage on every seed.
Scaled policy lengthscale ($\ell_s = 0.15 \times$ target range)	The paper's default $\ell_s=1.0$ makes basis functions so wide the policy can barely distinguish near from far targets — it converges to "max speed for everything" on unlucky seeds.	Closed-form rule from the target geometry; for Config A (range 1.0 m) $\rightarrow \ell_s=0.26$. No per-config hand-tuning.
PyBullet physics backend (KUKA iiwa7)	Adds real rigid-body arm dynamics (IK, joint limits, 3-phase throw trajectory) between "what the policy commands" and "what happens" — the gap that makes this an RL problem rather than an equation inversion.	IK + cubic joint trajectory (neutral $\rightarrow$ windup $\rightarrow$ release); ball velocity set at release; manual drag force per paper Eq. 35; landing interpolated to $z=0$.
Multi-seed protocol (seeds 1–5, fixed a priori)	Single-seed results in RL are existence proofs, not expected performance (Henderson et al., AAAI 2018). This project's own data proves it: the same config goes 0/10 or 10/10 depending on seed.	All seeds fixed before running; all reported, including failures. Evaluation targets identical across seeds for fair comparison.

2. Why the lengthscale rule works

An RBF policy can only produce different outputs for two targets if its basis functions *activate differently* on them. The similarity between activations at two targets distance d apart is $S = \exp(-d^2/2\ell s^2)$. With the paper's default $\ell s=1.0$, the two most distant targets in our domain still look 61% identical to the network — it is structurally near-blind to where the target is. The rule $\ell s = 0.15 \times \text{range}$ pushes S at full separation to ~ 0.001 : fully distinct.

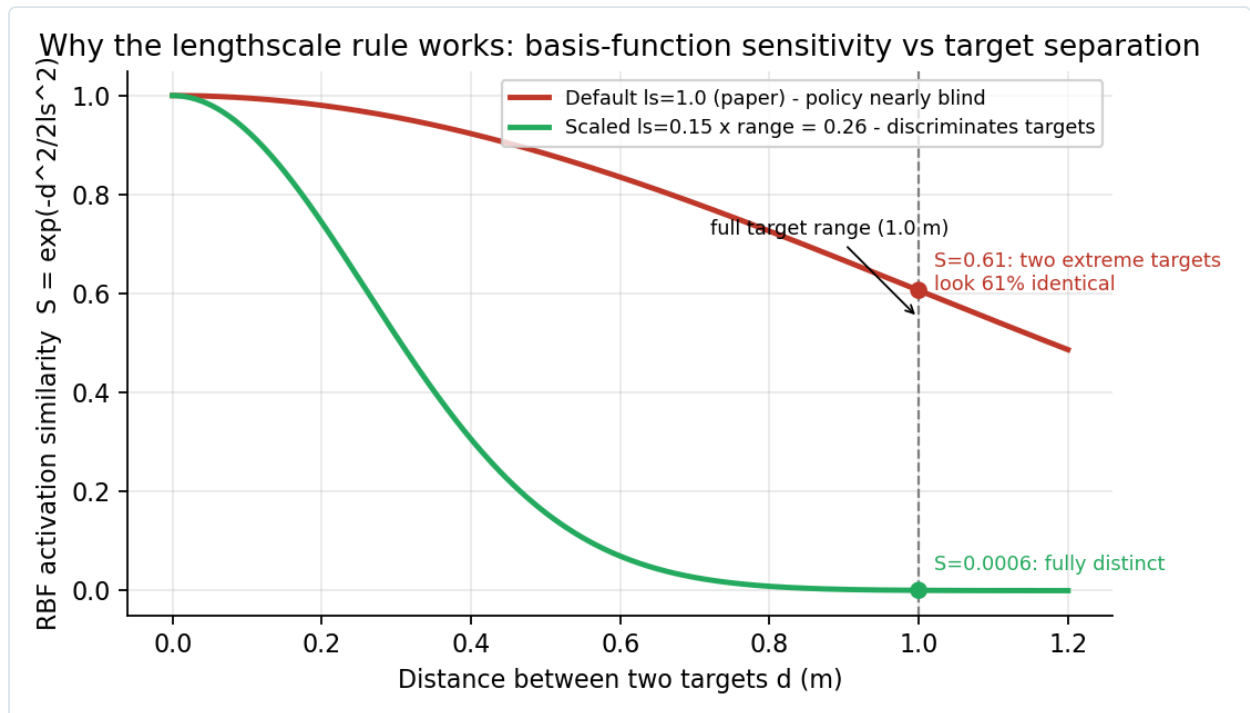


Fig 7 — Basis-function similarity vs target separation. Red: paper default (blind). Green: scaled rule (discriminating).

The consequence is directly visible in what the policy learns. Same seed, before and after:

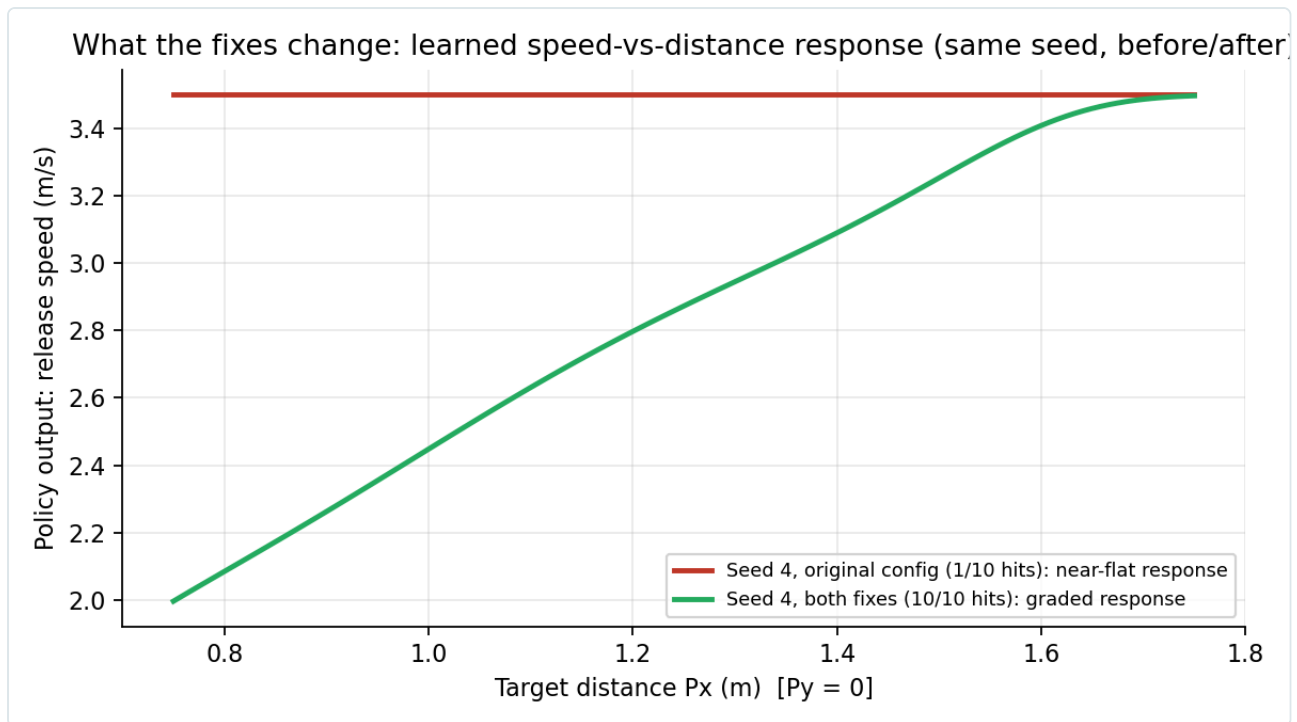


Fig 10 — Learned speed-vs-distance response. The failed configuration (red) outputs maximum speed for every target — a flat, target-blind policy (1/10 hits). With both fixes (green), the response is smooth and graded (10/10 hits).

3. Why stratified exploration works

The GP can only model dynamics it has seen. These are the *actual exploration speeds drawn* in our runs: with random draws, several seeds leave whole speed regions empty (e.g. seed 1 never threw above 2.7 m/s; seed 4's draws cluster mid-range) — the GP extrapolates blindly there, and the policy optimizer happily walks into the unmodeled region. Stratified draws cover every band on every seed.

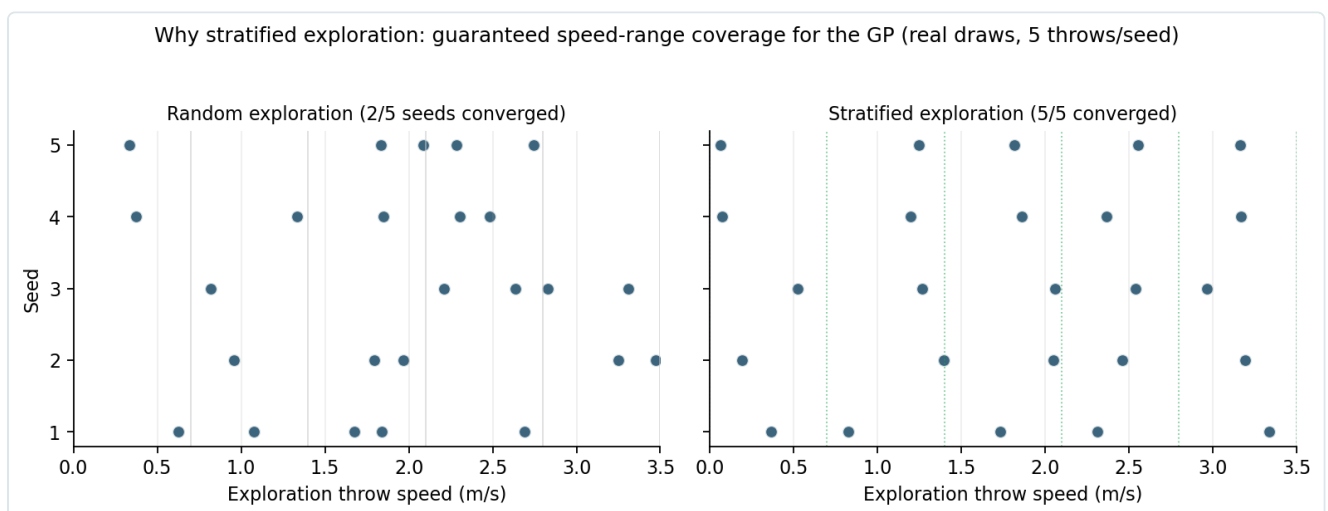


Fig 8 — Real exploration draws from the logs. Left: random (gaps → 2/5 converge). Right: stratified (full coverage → 5/5 converge).

4. Why the PyBullet fix works (and why the bug was invisible at first)

At release, the code re-enabled ball–arm collision while the ball still overlapped the swinging arm; the follow-through struck the ball, injecting contact impulses ($|\Delta v_z|$ up to $8\times$ gravity per step) into the first flight transitions of the training data. The failure was *gradual* because of how GP likelihood optimization treats outliers: with few contaminated points it classifies them as noise ($\sigma_n \approx 0.10$ — early trials look perfect); as they accumulate it starts *fitting* them — the Δv_z GP's lengthscale collapses $250 \rightarrow 8.4$ and $\sigma_n \rightarrow 0.006$, producing an overfit model whose rollout error explodes from 10^{-5} to 0.27 . The policy then optimizes against a hallucinating model.

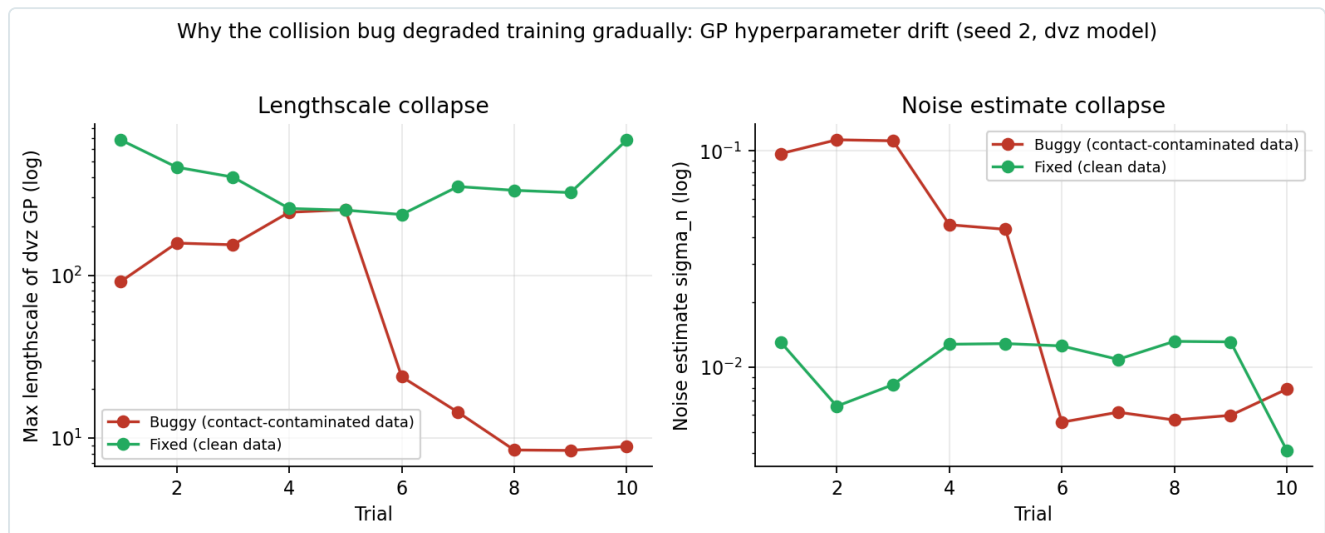


Fig 9 — GP hyperparameter trajectories, seed 2. Buggy data (red): lengthscale and noise collapse at trial 6 — exactly when cost blew up. Clean data (green): stable.

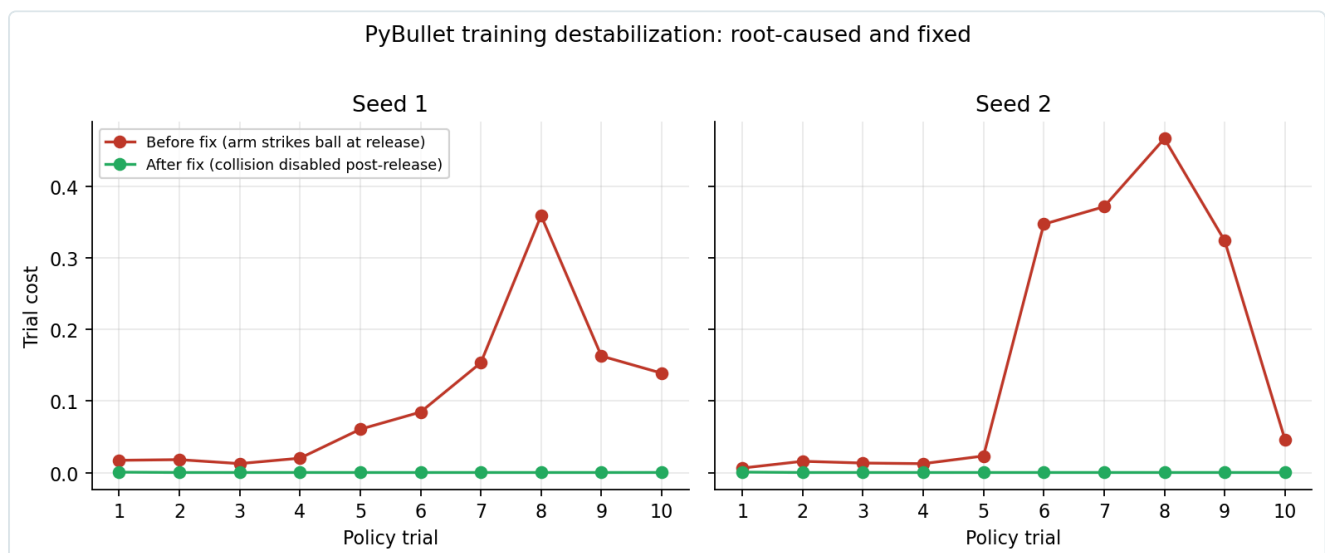


Fig 4 — Training cost per trial, seeds 1–2, before (red) vs after (green) the release-collision fix.

Fix: keep ball–arm collision disabled after release (2-line change). Justified: the codebase already sets release velocity explicitly (`resetBaseVelocity`) — the arm is cosmetic at release by design, so the strike was pure artifact. Verified post-fix: Δv training targets are pure gravity+drag physics.

5. Reliability results (why multi-seed was worth it)

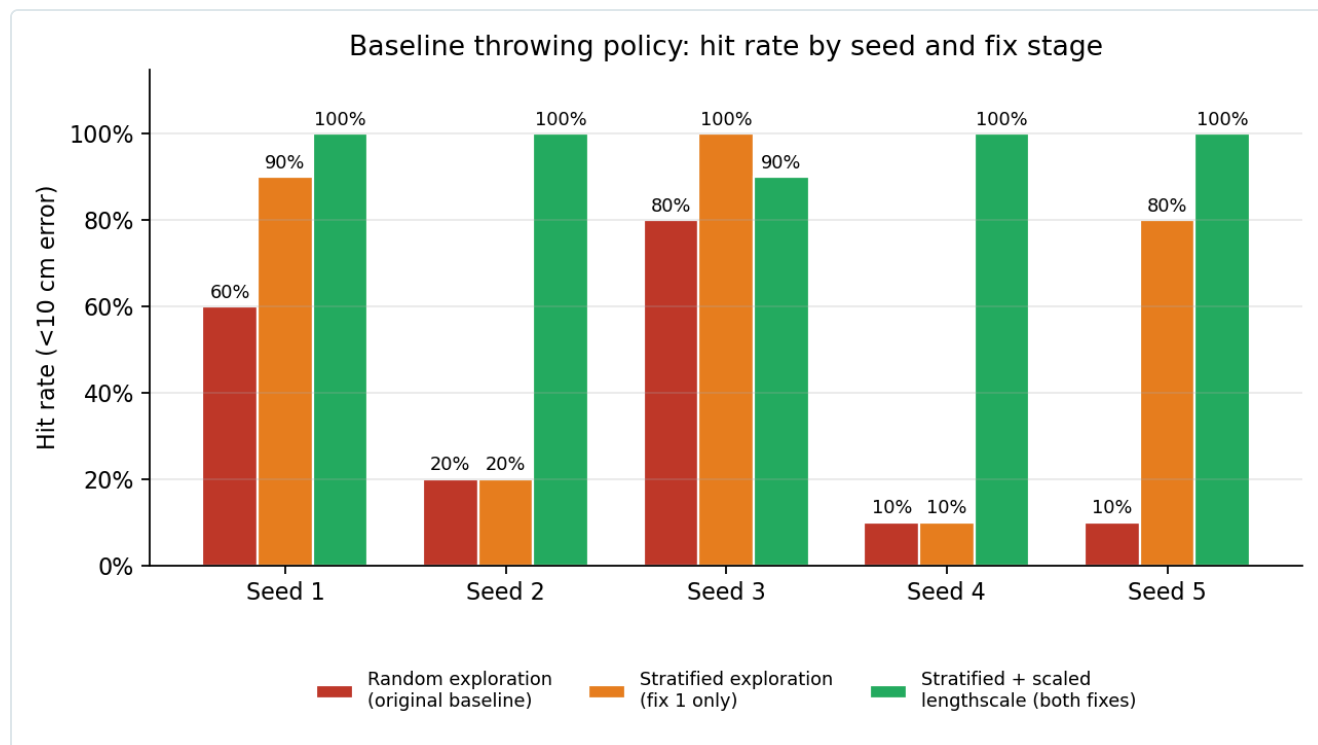


Fig 1 — NumPy baseline hit rate by seed across fix stages. The published single-seed "5/5" hides a 3/5-seed failure rate.

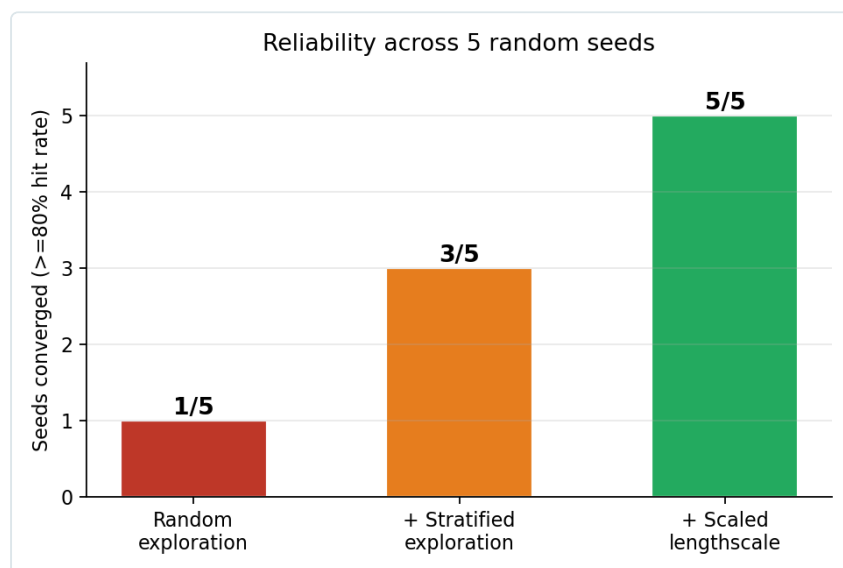


Fig 2 — Seeds converging: 1/5 (random) → 3/5 (stratified) → 5/5 (both fixes).

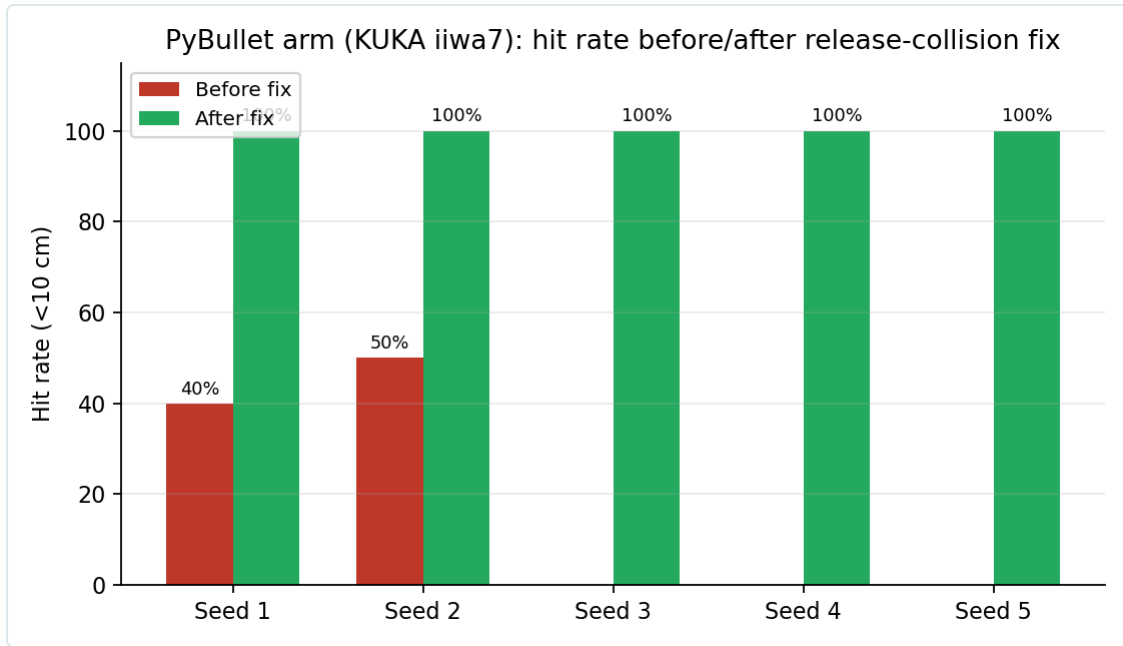


Fig 5 — PyBullet hit rates, before vs after the collision fix, all seeds.

6. Final evaluation (proper protocol)

Each trained seed evaluated on 50 **fresh** random targets (never seen in training), identical target set across seeds, through the exact training pipeline:

Seed	Targets	Hit <10cm	Hit <5cm	Mean err	Median	P95	Max
1	50	100%	100%	1.8 cm	1.6 cm	3.2 cm	3.8 cm
2	50	100%	100%	1.9 cm	1.9 cm	2.9 cm	3.3 cm
3	50	100%	100%	1.8 cm	1.6 cm	3.2 cm	3.7 cm
4	50	100%	100%	1.8 cm	1.7 cm	3.1 cm	3.6 cm
5	50	100%	100%	2.0 cm	2.0 cm	3.2 cm	3.7 cm
All	250	100%	100%	1.86 cm	1.77 cm	3.17 cm	3.81 cm

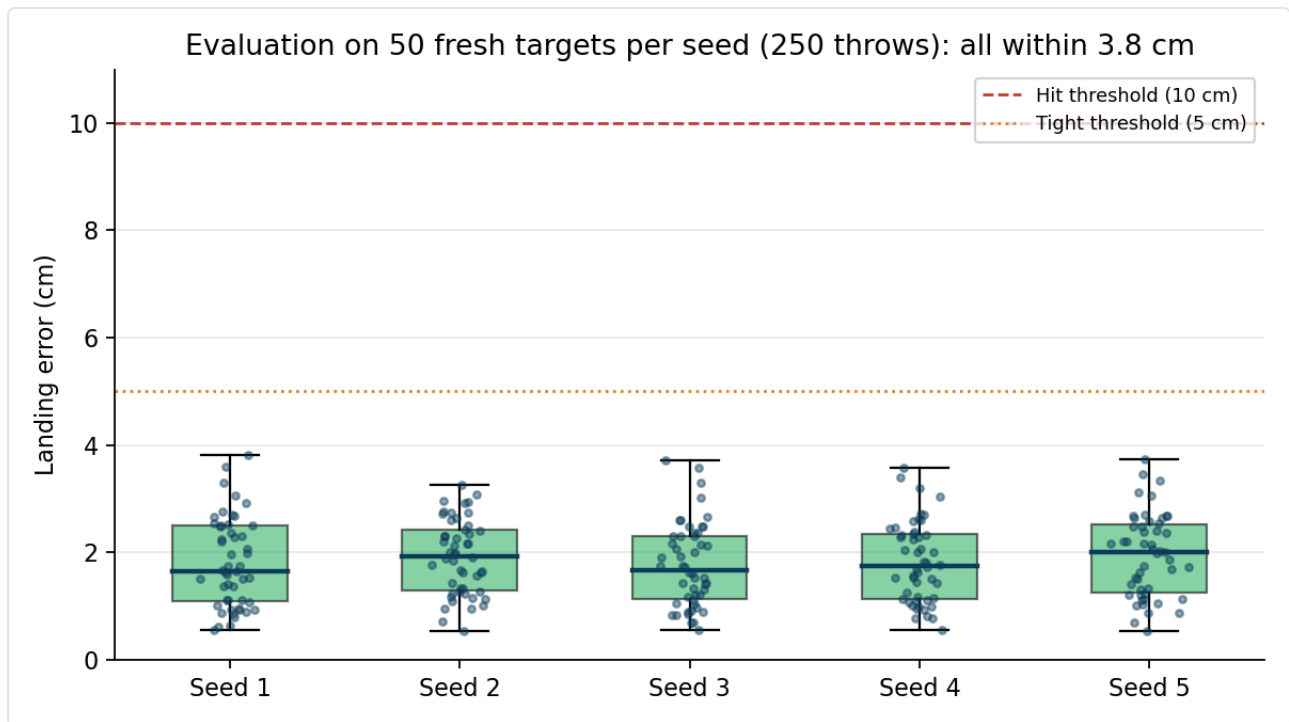


Fig 6 — Error distribution per seed, all 250 throws. Every point sits far below both thresholds.

7. Honest scope

These results are in a deterministic simulator with noise-free evaluation — they demonstrate that the learning pipeline is **correct, stable, and seed-robust**, not that it transfers to hardware. The noise-injection framework (velocity slip, timing jitter) and hardware validation are the stated next steps. Launch angle is fixed at 35°; the policy learns release speed only.

Companion files: mc_pilot_pb_A_demo.mp4 (6/6 hits, recorded from the exact training pipeline), eval_matrix.csv/.md, meeting_notes_pybullet_fix.md, meeting_prep.md.